

论文模板 05 · 系统安全架构设计

配套素材见 [00-项目素材库.md](#) 。本主题**稳定出现**，且与你的项目契合度高。

一、真题常见问法

变体 A（安全体系整体）

请论述你参与的某系统的安全架构设计，包括身份认证、访问控制、数据安全与安全审计等方面。

变体 B（认证授权）

请论述你如何设计系统的统一身份认证与授权体系，并说明如何实现细粒度权限控制。

变体 C（数据安全）

请论述你如何保障系统敏感数据的机密性、完整性与可用性。

判断：BASE HOME 的最强项是**变体 A 与变体 B**——项目真实使用了 OAuth2/OIDC、Spring Security、细粒度 scope 权限、Redis Session、STS 临时凭证、TLS 传输。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的安全架构设计与实现。

该平台承载旅行者的护照信息、亲权同意书（親権同意書）、紧急联系人等敏感个人数据，且需与集团多个外部系统交互，安全要求高、合规约束严。

本人担任架构师，负责身份认证体系设计、权限模型设计与凭证管理方案。

针对【多角色细粒度权限控制】、【跨系统凭证泄露风险】、【敏感数据传输与存储安全】三大挑战，

本文从【统一身份认证】、【基于 scope 的细粒度授权】、【临时凭证与加密传输】三个维度进行设计：

基于 Spring Security 集成 OAuth2/OIDC 实现统一登录，

以权限 scope 列表驱动接口级授权，

以 STS AssumeRole 临时凭证替代长期密钥并与 TLS 结合保障传输安全。

系统上线后【 填效果】，验证了方案的可行性。

三、正文五段框架

第 1 段 · 项目背景与安全需求 (约 400 字)

安全论文的开头必须写「保护什么」，否则后面所有措施都失去依据。

BASE HOME 处理的数据具有高度敏感性，我将其分级为三类：① **个人身份信息**——旅行者护照信息、姓名、出生日期；② **特殊敏感数据**——未成年人亲权同意书（親権同意書）、紧急联系人信息；③ **业务敏感数据**——预约成本、客户消息内容。此外，系统需与集团 MDM 主数据平台、外部预约系统、邮件网关、消息推送服务等多方交互，每个交互面都是潜在的攻击入口。合规方面，系统需满足日本个人信息保护法的相关要求。

第 2 段 · 架构设计 (约 800 字)

按安全架构的经典四层组织：身份认证 → 访问控制 → 数据安全 → 安全审计

第一层：身份认证 (Authentication)

系统采用 OAuth2/OIDC 协议实现统一身份认证，Web 层通过 `spring-boot-starter-oauth2-client` 接入集团统一认证服务，以授权码模式完成登录，令牌校验使用 `spring-security-oauth2-jose` (JOSE/JWT)。用户身份属性通过 `user-name-attribute` 配置从令牌中提取，避免依赖本地用户表——这使账号生命周期由集团统一治理。

同时通过 `thymeleaf-extras-springsecurity6` 把安全上下文暴露到视图层，使页面元素的显隐可以直接由权限决定，而不是靠前端 JavaScript 隐藏（后者只是「看不见」，后者才能做到「点不到」）。

权衡说明：OIDC 引入了对外部认证服务的依赖，我们通过将用户信息缓存于会话中，使认证服务短暂不可用时已登录用户仍可继续操作，避免认证服务成为可用性单点。

第二层：访问控制 (Authorization)

授权模型我采用 **RBAC + 接口级 scope 校验** 的组合。角色维度划分为管理员 (admin)、操作员 (operator)、主管 (chief)、VIP 等，各角色持有不同的权限 scope 列表。权限校验下沉到接口层（`/api/v1/**` 由 Spring Security 统一拦截），而非散落在各 Controller 的业务代码中——这样新增接口时权限是「默认拒绝」而非「默认放行」，从架构上避免了「忘记加权限注解」这类遗漏风险。

第三层：数据安全 (Data Security)

传输安全：内部服务间 gRPC 调用使用 TLS (endpoint 采用 `tls://` 前缀)，对外 Web 访问全站 HTTPS。**凭证安全：**这是我重点处理的部分。系统不使用长期固定的 AWS 密钥，而是通过 **STS AssumeRole 获取临时凭证**，配合 `DefaultCredentialsProvider` 兜底，凭证自动轮换，即使泄露其有效期也极短。**配置安全：**FaaS 侧所有配置（数据库连接、外部端点）统一从 SSM Parameter Store 读取，代码仓库中不落任何明文凭证。**存储安全：**文件实体统一存放 S3，数据库仅保存文件引用；数据库账号按 reader/writer 职责分离，批处理读账号无写权限。

第四层：安全审计 (Audit)

系统在 Web 层设置独立的审计能力：画面操作日志由 `system` 域的控制器统一记录，关键业务操作（预约变更、消息发送、主数据修改）均留存操作者、操作时间与操作内容。审计日志与业务数据分离存储，避免业务回滚时日志同时丢失。

第 3 段 · 关键技术实现 (约 700 字)

要素一：会话安全

会话由 **Spring Session Data Redis** 统一管理，存储于 **Redis Cluster (6 节点)**，超时时间 3600 秒。相比本地会话，集中式会话带来两个安全收益：一是会话数据不落在应用服务器磁盘上，降低主机被入侵后的会话窃取风险；二是可在服务端强制失效任意会话，实现「管理员踢出用户」与「用户改密后全端下线」这类安全能力。

要素二：凭证与密钥的集中治理

我把凭证获取统一收口到配置类（`DynamoDbConfig`、`AwsConfig`、`SsmClient`），业务代码不直接接触凭证。这样做的价值在于：凭证轮换、加密算法升级、权限范围调整都只需在单一位置修改，不会遗漏调用点。环境隔离同样重要——DynamoDB 表名统一带环境前缀，实现「1 账户 1 集群」，防止测试环境误连生产数据。

要素三：接口契约中的安全边界

由于系统通过 `protobuf` 向 Java/Go/PHP 三端提供接口，契约本身也承担安全职责：每个 RPC 均声明 HTTP 映射（`google.api.http`），统一走 `/v1/<RpcName>` 路径，便于在网关层集中做鉴权与限流；错误信息通过统一的 `Error{code,message}` 结构返回，对外只暴露错误码与友好提示，**不泄漏堆栈与内部实现细节**。

要素四：输入校验与注入防护

Web 层启用 `spring-boot-starter-validation` 做参数校验，数据库访问全部通过 MyBatis 参数绑定，杜绝字符串拼接 SQL。涉及 HTML 内容处理的场景使用 Jsoup 做净化，防止存储型 XSS——这在客户消息场景尤其重要，因为消息内容会在 MyPage 页面回显给其他用户。

第 4 段 · 难点与解决 (约 700 字)

安全主题最佳组合（安全主题的难点需要自行组织，素材库难点适配度中等）：

优先级	难点方向	素材来源
首选	多角色细粒度权限的默认拒绝设计	由项目 scope 权限模型推导
次选	三语言契约下的安全边界统一	素材库 ①

优先级	难点方向	素材来源
三选	跨国多时区与数据合规	素材库 ⑧
加分	凭证长期化风险治理	素材库 ③ (v1/v2 SDK 并存背景下的凭证统一)

难点示例（权限默认拒绝）：

问题：系统角色众多（管理员、操作员、主管、VIP 等），且不同角色可见的业务域差异极大。项目初期采用在 Controller 方法上 逐个添加权限注解的方式，随着接口数量增长（REST 接口按业务域 分布在 8 个包下），出现了「新增接口忘记加注解」的遗漏风险——这类漏洞不会在测试期暴露，只在被利用时才显现。

方案：我把授权策略从「逐个放行」改为「默认拒绝」。在 Spring Security 配置中统一拦截 `/api/v1/**`，未显式声明的路径一律拒绝；权限以 scope 列表的形式 随令牌下发，接口层按 scope 校验而非按角色硬编码判断。这样角色与权限的关系可以在配置侧调整，无需修改业务代码——新增角色只需分配 scope 组合。

效果：◆ 消除新增接口的权限遗漏风险；角色调整从「改代码」变为「改配置」，◆ 权限变更耗时从 X 降到 Y。

第 5 段 • 总结与不足（约 200 字）

目前该安全架构已稳定支撑【◆ 用户规模】，未发生【◆ 安全事件情况】。

不足之处：

一是权限模型目前以角色为单位的 scope 组合为主，尚未实现对单条数据级（行级）的细粒度控制；

二是跨语言契约的破坏性变更尚未引入自动化安全影响评估。

后续计划引入数据级授权与契约变更审计。

通过本项目我认识到，安全架构的核心不是堆砌安全产品，而是建立“默认拒绝”的基线，并让凭证、权限、审计都收口到可治理的位置。

四、本项目填充点速查

论文要素	直接可写的内容
认证协议	OAuth2 / OIDC 授权码模式
认证实现	<code>spring-boot-starter-oauth2-client</code> + <code>spring-security-oauth2-jose</code>
身份来源	令牌中的 <code>user-name-attribute</code> ，账号由集团统一治理
视图层安全	<code>thymeleaf-extras-springsecurity6</code> （页面元素按权限显隐）
授权模型	角色 + scope 权限列表；接口层统一拦截 <code>/api/v1/**</code>
角色划分	admin / operator1~4 / chief / vip

论文要素	直接可写的内容
会话管理	Spring Session Data Redis + Redis Cluster (6 节点) + Jedis, 3600s
传输安全	gRPC 使用 <code>tls://</code> endpoint; 对外 HTTPS
凭证管理	STS AssumeRole 临时凭证 + <code>DefaultCredentialsProvider</code>
配置安全	SSM Parameter Store (FaaS) ; 凭证集中在 <code>DynamoDbConfig</code> / <code>AwsConfig</code>
环境隔离	DynamoDB 表名带环境前缀, 「1 账户 1 集群」
数据库权限	reader / writer 双数据源职责分离
审计	<code>system</code> 域画面操作日志 (操作者/时间/内容)
输入校验	<code>spring-boot-starter-validation</code> ; MyBatis 参数绑定防注入
内容净化	Jsoup (1.22.1) 处理 HTML 内容, 防存储型 XSS
错误信息	统一 <code>Error{code,message}</code> , 不泄漏内部细节
消息规范	消息 ID 前缀区分类型 (E=错误、F=字段标签)
接口文档	SpringDoc OpenAPI <code>/api-docs</code> , 仅匹配 <code>/api/v1/**</code>
日志脱敏	properties 自动生成消息常量, 避免硬编码敏感文案

💡 「消息 ID 体系」是一个容易被忽略的亮点：系统通过 properties 文件自动生成消息常量类（`ApiMessageId`、`WebMessageId`、`FileItem`，均标注「自动生成、修正不可」），并以 ID 首字符区分消息类型。这既是代码规范，也是安全审计与国际化的双重基础设施——写安全或可维护性时都可以用。

五、可用难点素材（按适配度排序）

- 1. 权限模型的「默认拒绝」重构 ★★★★★ —— 本主题首选（需自行组织，参考上文示例）
- 2. ① 一份 proto 契约驱动三语言 ★★ —— 契约级安全边界统一
- 3. ③ AWS SDK v1/v2 并存下的凭证统一 ★★ —— 凭证治理
- 4. ⑧ 跨国多时区一致性 ★★ —— 数据合规维度
- 5. ⑨ 分布式会话 Redis Cluster ★★ —— 会话安全

六、踩坑提醒

坑	后果	正确做法
罗列安全产品	空洞	按「保护什么 → 威胁是什么 → 措施是什么」组织
编造等保/ISO 认证	高风险	未确认的合规资质不要写；写「参照日本个人信息保护法要求」更稳妥
编造加密算法细节	高风险	只写「TLS 传输加密」「STS 临时凭证」，不编 AES-256/国密等具体算法

坑	后果	正确做法
编造 WAF/堡垒机/IDS	高风险	本项目未见此类证据，不要写
忽略审计	漏考点	安全架构四要素「认证/授权/加密/审计」缺一不可
只说技术不说管理	片面	补上凭证轮换、环境隔离、权限变更流程等治理手段
忘记「默认拒绝」原则	关键失分	这是安全架构最重要的设计原则，务必体现

⚠ **技术真实性提醒：** 本项目有 **OAuth2/OIDC + Spring Security + TLS + STS 临时凭证 + Jsoup 净化**，但**未见** WAF、IDS/IPS、堡垒机、加密机、国密算法等企业级安全设备的使用证据。论文中不要为了「显得安全」而堆砌未使用的产品。相反，把「默认拒绝的授权设计」「凭证集中治理」这两个**真实的架构决策**写透，比列十个安全产品更有说服力。

配套： [01- 微服务架构设计.md](#) · [06- 系统集成与gRPC.md](#) (契约安全部分)